

Top 20 Real-World DevOps Interview Questions

Structured, Point-by-Point Answers for Interview Preparation

Q1. Tell me about a production incident you handled and how you resolved it.

Suggested Answer Framework (STAR Method):

- **Situation:** Briefly describe the system and the impact (e.g., "Payment service started returning 500 errors during peak traffic").
- **Task:** State your role and responsibility in resolving it (on-call engineer, lead responder).
- **Action:** Explain the diagnostic steps — checked logs/metrics (CPU, memory, error rate), reviewed recent deployments, isolated the faulty service/pod.
- **Resolution:** Describe the fix — rollback, scaling up, restarting a service, patching a config, or applying a hotfix.
- **Result:** Quantify the outcome (downtime reduced, MTTR, customer impact minimized).
- **Follow-up:** Mention the post-incident review (RCA), and the preventive action taken (alerting rule, runbook, auto-scaling policy).

Key takeaway interviewers look for:

- Calm, structured troubleshooting under pressure, clear communication, and learning-driven follow-up — not just the fix itself.

Q2. How would you design a CI/CD pipeline for a microservices application?

- **Source stage:** Each microservice has its own Git repository (or monorepo with path-based triggers) with branch protection rules.
- **Build stage:** Containerize each service with Docker; use multi-stage builds for smaller images; run unit tests and static code analysis (SonarQube, lint).
- **Test stage:** Run integration tests against ephemeral environments; include security scanning (SAST/DAST) and dependency vulnerability checks.
- **Artifact stage:** Push versioned images to a container registry (ECR, Docker Hub, GCR) with semantic tagging.
- **Deploy stage:** Use Helm charts or Kustomize to deploy to Kubernetes; promote through environments (dev → staging → production) using GitOps (ArgoCD/Flux).
- **Release strategy:** Apply blue-green or canary deployment for safer rollouts; automated rollback on failed health checks.
- **Observability:** Integrate monitoring, logging, and tracing into the pipeline gates (e.g., fail deploy if error rate spikes post-release).
- **Independence:** Ensure each service can be built, tested, and deployed independently to preserve microservices autonomy.

Q3. What is the difference between CMD and ENTRYPOINT?

Aspect	CMD	ENTRYPOINT
Purpose	Provides default arguments/command	Defines the fixed executable that always runs
Overridable	Easily overridden by command-line arguments at <code>docker run</code>	Not overridden by default; arguments append to it (unless <code>--entrypoint</code> is used)
Use case	Best when the container's command may change	Best when the container should always run as a specific binary (e.g., an app)
Combined use	CMD supplies default args to ENTRYPOINT	ENTRYPOINT executes; CMD's values become its arguments

Example:

```
ENTRYPOINT ["python3"]
```

```
CMD ["app.py"] → runs as: python3 app.py
```

Q4. How do you manage secrets and sensitive information in your deployments?

- Never hard-code secrets in source code, Dockerfiles, or plain CI/CD variables.
- Use a dedicated secrets manager: **HashiCorp Vault**, **AWS Secrets Manager**, **Azure Key Vault**, or **GCP Secret Manager**.
- In Kubernetes, use **Kubernetes Secrets** (encrypted at rest with KMS) or integrate **External Secrets Operator**/Vault injector.
- Apply **least-privilege IAM roles** so only required services/pods can read specific secrets.
- Rotate secrets and credentials regularly and automate rotation where possible.
- Mask secrets in CI/CD logs and use encrypted pipeline variables (e.g., GitHub Actions secrets, GitLab CI variables).
- Enable audit logging on secret access for traceability and compliance.

Q5. What are Docker layers and their significance?

- Each instruction in a Dockerfile (FROM, RUN, COPY, ADD, etc.) creates a new **read-only layer** stacked on top of the previous one.
- Layers are cached — if a layer hasn't changed, Docker reuses it on rebuild, speeding up builds significantly.
- The final container adds a thin **writable layer** on top of the read-only image layers for runtime changes.

Significance:

- **Faster builds:** unchanged layers are pulled from cache.
- **Smaller, reusable images:** common base layers can be shared across multiple images.
- **Efficient storage and transfer:** only changed layers need to be pushed/pulled from a registry.
- **Best practice:** order Dockerfile instructions from least-to-most frequently changing (e.g., install dependencies before copying source code) to maximize cache reuse.

Q6. How would you perform a zero-downtime deployment?

- **Rolling updates:** Replace old pods/instances gradually while keeping enough healthy instances to serve traffic (default Kubernetes Deployment strategy).
- **Blue-Green deployment:** Run two identical environments; switch traffic at the load balancer/router once the new version passes health checks.
- **Canary deployment:** Release the new version to a small percentage of users first, monitor metrics, then gradually increase traffic.
- **Readiness and liveness probes:** Ensure traffic is only routed to pods that are fully ready.
- **Database migration strategy:** Use backward-compatible schema changes so old and new versions can run simultaneously during the cutover.
- **Session/connection draining:** Allow in-flight requests to complete before terminating old instances.
- **Automated rollback:** Trigger an immediate rollback if error rates or latency spike post-deployment.

Q7. What steps would you take if a Kubernetes pod keeps restarting?

- Check pod status and restart count: `kubectl get pods`
- Describe the pod for events and reasons: `kubectl describe pod <pod-name>`
- View current and previous container logs: `kubectl logs <pod> --previous`
- Identify the failure type: **CrashLoopBackOff** (app crashing), **OOMKilled** (memory limit exceeded), or failing **liveness/readiness probes**.
- Check resource requests/limits — insufficient memory/CPU often causes restarts.
- Verify environment variables, config maps, and secrets are correctly mounted.
- Check application-level errors (bad configuration, missing dependency, failed DB connection).
- If needed, exec into a debug pod or temporarily increase resource limits to isolate the issue.
- Fix root cause, redeploy, and monitor pod stability before closing out.

Q8. How do you troubleshoot a failed deployment in production?

- **Confirm the failure scope:** Is it affecting all users, a region, or a specific service?
- **Check deployment logs/events** in the CI/CD tool and orchestration platform (Kubernetes events, ECS task status, etc.).
- **Review recent changes:** compare the failing deployment against the last known-good version (diff configs, code, infra).
- **Check application and infrastructure metrics:** CPU, memory, error rate, latency, dependency health.
- **Validate external dependencies:** databases, third-party APIs, message queues.
- **Roll back immediately** if the issue is severe and the root cause isn't quickly identifiable, to restore service.
- **Perform root cause analysis** after stabilization and document findings in a post-mortem.
- **Implement preventive measures:** better pre-deployment tests, canary checks, or alerting thresholds.

Q9. Explain the difference between Blue-Green and Canary deployments.

Aspect	Blue-Green	Canary
Approach	Two full identical environments; traffic switched all at once	New version rolled out to a small subset of traffic first
Risk exposure	All-or-nothing cutover (fast rollback by switching back)	Gradual exposure, limits blast radius of issues
Infrastructure cost	Higher — requires duplicate environment	Lower — same environment, traffic-split based
Rollback speed	Very fast (just re-route traffic)	Fast, but only affects the canary slice
Best for	Critical releases needing instant rollback capability	Validating new features/performance with real user traffic gradually

Q10. How would you design a highly available architecture on AWS?

- Deploy across **multiple Availability Zones** (AZs) within a region; use multiple regions for disaster recovery if required.
- Use an **Auto Scaling Group** behind an **Application Load Balancer (ALB)** to distribute traffic and handle instance failures.
- Use managed, multi-AZ databases: **RDS Multi-AZ** or **Aurora** with read replicas.
- Use **Route 53** with health checks for DNS failover, and **CloudFront** for content delivery and edge caching.
- Store static assets/state in **S3** (which is inherently multi-AZ and highly durable).
- Decouple components with **SQS/SNS** to absorb traffic spikes and prevent cascading failures.
- Implement **health checks, circuit breakers, and retries** at the application layer.

- Use Infrastructure as Code (Terraform/CloudFormation) so the entire stack is reproducible in another region if needed.
- Continuously monitor with **CloudWatch** and set up automated alerting and self-healing (Auto Scaling replacing unhealthy instances).

Q11. Secrets are accidentally pushed to Git. How do you handle the situation?

- **Immediately rotate/revoke** the exposed credentials (API keys, passwords, tokens) — this is the most urgent step, regardless of repo visibility.
- **Remove the secret from git history**, not just the latest commit — use tools like `git filter-repo` or BFG Repo-Cleaner to purge it from all commits.
- **Force-push** the cleaned history and notify all collaborators to re-clone or rebase.
- If pushed to a public repository, assume the secret is **compromised permanently**, even after removal (it may be cached/indexed elsewhere).
- Audit logs/systems for any unauthorized use of the leaked credential.
- Add preventive controls: **pre-commit hooks** (git-secrets, talisman), **.gitignore** rules, and CI pipeline secret-scanning (GitGuardian, TruffleHog).
- Move sensitive values to a proper secrets manager going forward instead of storing them in code or config files.

Q12. How do you integrate monitoring tools into applications and Kubernetes clusters?

- **Metrics:** Deploy **Prometheus** to scrape application and cluster metrics; expose custom metrics via client libraries (e.g., `prometheus_client`).
- **Visualization:** Use **Grafana** dashboards built on Prometheus/CloudWatch data sources.
- **Cluster-level monitoring:** Use **kube-state-metrics** and **node-exporter** for pod, node, and resource-level visibility.
- **Logging:** Centralize logs with the **EFK/ELK stack** (Elasticsearch, Fluentd/Fluent Bit, Kibana) or a managed service (CloudWatch Logs, Loki).
- **Tracing:** Implement distributed tracing with **Jaeger** or **OpenTelemetry** for cross-service request visibility.
- **Alerting:** Configure **Alertmanager** or cloud-native alerting (CloudWatch Alarms) integrated with Slack/PagerDuty/Opsgenie.
- **Health checks:** Use readiness/liveness probes feeding into monitoring for automated remediation triggers.

Q13. What would you do if your CI/CD pipeline suddenly started failing?

- Check the **pipeline logs** to identify the exact failing stage (build, test, deploy).
- Determine if the failure is **environment-related** (e.g., runner/agent issue, expired credentials, network/registry outage) or **code-related** (failing tests, broken dependency).
- Check if a recent change to the **pipeline configuration** (YAML, scripts) or a dependency version bump caused it.
- Reproduce the failure locally or in an isolated environment if possible.
- Check third-party service status (artifact registry, package repository, cloud provider) for outages.
- Apply a fix or temporarily pin/rollback a dependency version to restore pipeline stability.

- Once resolved, document the root cause and add a safeguard (version pinning, additional validation step, better error messages).

Q14. Explain the complete request flow from a user accessing an application to receiving a response.

- **1. DNS resolution:** User's browser resolves the domain name via DNS (e.g., Route 53) to an IP address.
- **2. CDN/Edge layer:** Request may be served or cached at a CDN edge location (CloudFront, Akamai) for static content.
- **3. Load Balancer:** Request hits a Load Balancer (ALB/NLB or Ingress Controller) which terminates TLS and routes traffic.
- **4. Ingress/API Gateway:** Routes the request to the appropriate service based on path/host rules; may handle authentication, rate limiting.
- **5. Application/Service layer:** Request reaches the relevant microservice (running in a pod/container), which processes business logic.
- **6. Service-to-service calls:** The service may call other internal services via a service mesh or internal load balancer.
- **7. Data layer:** Application queries a database, cache (Redis), or message queue as needed.
- **8. Response path:** The response travels back through the same chain — service → load balancer → CDN (if cached) → user's browser.
- **9. Observability:** Throughout this flow, logs, metrics, and traces are captured for monitoring and debugging.

Q15. How do you secure containerized applications?

- Use **minimal base images** (e.g., Alpine, distroless) to reduce attack surface.
- Scan images for vulnerabilities using tools like **Trivy**, **Clair**, or **Snyk** before deployment.
- Run containers as a **non-root user** and apply read-only file systems where possible.
- Avoid storing secrets in images; use secret managers and inject at runtime.
- Apply **Kubernetes security best practices**: Network Policies, Pod Security Standards/Admission, RBAC with least privilege.
- Keep images and dependencies **patched and updated** regularly.
- Sign and verify images (e.g., **cosign**/Notary) to ensure supply chain integrity.
- Restrict container capabilities (drop unnecessary Linux capabilities, disable privileged mode).
- Continuously monitor runtime behavior with tools like **Falco** for anomaly detection.

Q16. What are Terraform state files, and how do you manage them in a team environment?

What is a state file:

- The Terraform state file (`terraform.tfstate`) is a JSON file that maps real-world infrastructure resources to your configuration, tracking metadata like resource IDs and attributes.

Team management best practices:

- Use a **remote backend** (S3 + DynamoDB, Terraform Cloud, Azure Storage, GCS) instead of local state files.
- Enable **state locking** (e.g., DynamoDB lock table) to prevent concurrent modifications and corruption.

- Enable **encryption at rest** for the state file since it may contain sensitive data.
- Use **workspaces** or separate state files per environment (dev/staging/prod) to isolate changes.
- Restrict access via IAM policies — only authorized pipelines/users should read/write state.
- Version and back up state regularly; enable versioning on the S3 bucket storing it.
- Review changes via `terraform plan` in CI before any apply, to avoid unreviewed drift.

Q17. What problems occur if you change the Terraform state file manually?

- **State/infrastructure drift:** The state no longer accurately reflects real infrastructure, causing Terraform to make incorrect decisions on the next apply.
- **Resource duplication or deletion:** Terraform may try to recreate resources it thinks don't exist, or destroy resources it thinks are orphaned.
- **Corrupted state file:** Manual JSON edits can introduce syntax errors, making the state unreadable and blocking all future operations.
- **Loss of dependency tracking:** Terraform's resource graph relies on accurate state; manual edits can break dependency relationships.
- **Team-wide impact:** Since state is shared, a bad manual edit can break Terraform runs for the entire team, not just one user.
- **Recommended approach instead:** Use safe, supported commands such as `terraform state mv`, `terraform state rm`, `terraform import`, or `terraform taint` rather than editing the file directly.

Q18. What strategies do you use for backup and disaster recovery?

- Define **RTO (Recovery Time Objective)** and **RPO (Recovery Point Objective)** based on business requirements first.
- Take **automated, regular backups** of databases and critical state (snapshots, dumps) with retention policies.
- Store backups in a **different region/location** than production to survive regional failures.
- Use **Infrastructure as Code** so entire environments can be rebuilt quickly in a new region if needed.
- Implement **multi-region or multi-AZ replication** for critical data stores (e.g., database read replicas).
- Regularly **test recovery procedures** (DR drills) — a backup that hasn't been restored and verified isn't reliable.
- Maintain a documented **runbook** for disaster recovery, including roles, communication plan, and step-by-step recovery process.
- Use **chaos engineering** practices to proactively validate resilience before a real disaster occurs.

Q19. How do you prioritize tasks during a critical production outage?

- **Stabilize first, diagnose later:** Focus on restoring service (rollback, failover, scaling) before doing deep root-cause analysis.
- **Assess impact and severity:** Determine how many users/services are affected and classify the incident severity (P1/P2/etc.).
- **Communicate early and often:** Notify stakeholders, open an incident channel, and assign a clear incident commander.
- **Delegate effectively:** Split tasks — one person investigates logs/metrics, another handles communication, another implements the fix.
- **Avoid risky changes mid-incident:** Don't introduce new, untested changes during an active outage unless necessary.
- **Document actions in real time** for the post-incident review.

- **Conduct a blameless post-mortem** afterward to identify root cause and preventive actions.

Q20. If you had to improve the existing DevOps process in an organization, what would you change first?

- **Assess current maturity first:** Review existing CI/CD, monitoring, and deployment practices to identify the biggest bottleneck.
- **Prioritize by impact:** Common high-impact starting points include:
 - Reducing deployment friction (manual steps → automated pipelines)
 - Improving observability (adding proper monitoring/alerting if missing)
 - Strengthening security practices (secret management, image scanning)
 - Introducing Infrastructure as Code if infrastructure is still manually managed
- **Start small, show value quickly:** Pick one impactful, low-risk improvement to build trust before larger changes.
- **Involve the team:** Gather input from developers and operations to ensure the change is adopted, not resisted.
- **Measure outcomes:** Use metrics like deployment frequency, lead time, change failure rate, and MTTR (the DORA metrics) to validate improvement.
- **Iterate continuously:** Treat DevOps improvement as an ongoing process, not a one-time fix.